T.C SAKARYA ÜNİVERSİTESİ

Bilgisayar ve Bilişim Bilimleri Fakültesi

BİLİŞİM SİSTEMLERİ MÜHENDİSLİĞİ ANA BİLİM DALI

KAPSAMLI PROJE RAPORU: Agri-Stress - XAI Destekli Bitki Stresi Teşhis ve Karar Destek Sistemi

ÖĞRETİM ÜYESİNİN UNVANI ADI-SOYADI

Dr.Öğr.Üyesi BARAN KAYNAK

Ders adı

ISE513 BULUT BİLİŞİM

ÖĞRENCİNİN ADI-SOYADI

HAITHM AMEEN SALEM BA AUOZAR

NUMARASI

Y255052084

SAKARYA, 2026 Baha

İçerikler

1. Giriş ve Kavramsal Çerçeve 3
2. Sistem Mimarisi ve Bileşenlerin Derinlemesine Analizi 3
2.1. Sunum Katmanı (Frontend) 4
2.2. API Ağ Geçidi ve Orkestratör (.NET 10) 4
2.3. Yapay Zeka ve XAI Motoru (Python FastAPI) 5
3. Konteynerizasyon ve Docker Mimari Kararları 5
3.1. Çok Aşamalı İnşa (Multi-stage Build - .NET API) 6
3.2. Docker Compose ve Bağımlılık Yönetimi 6
4. Bulut Orkestrasyonu: Kubernetes (K8s) Bildirimleri 6
5. Gözlemlenebilirlik: Prometheus ve Grafana Entegrasyonu 7
5.1. Prometheus Metrikleri ve Matris Veri Modeli (Matrix Data Model) 7
5.2. Grafana Gösterge Paneli (Dashboard) 8
6. Ölçeklendirme Stratejisi (Scaling Strategy) 8
7. Sonuç ve Öğrenilen Dersler 9
8. Kaynakça 10

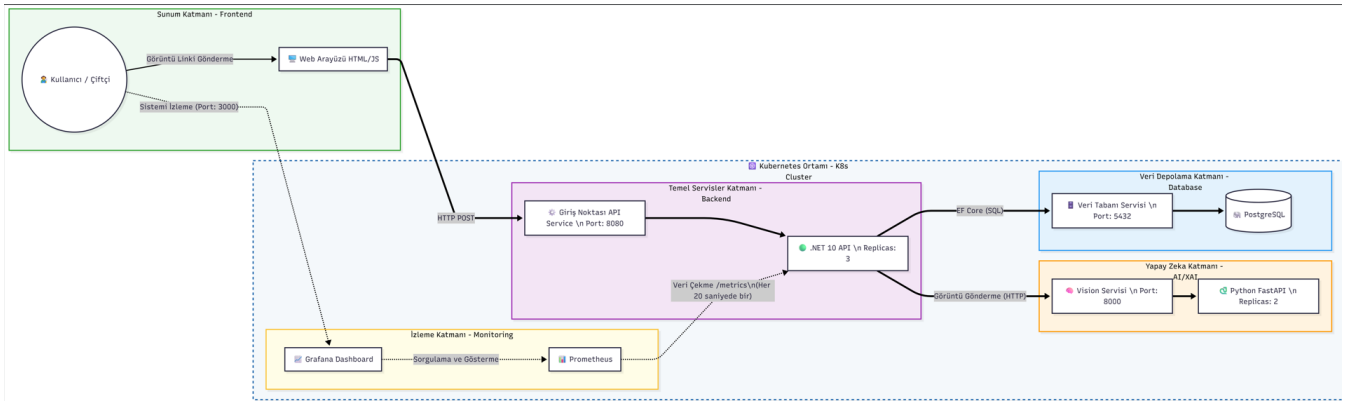
Proje Seçeneği: Seçenek 1 (Agri-Stress)

1. Giriş ve Kavramsal Çerçeve

Modern yazılım mühendisliğinde, uygulamaların ölçeklenebilirliği, esnekliği ve hata toleransı (fault tolerance) kritik bir öneme sahiptir. Geleneksel Monolitik Mimari, tüm bileşenlerin tek bir işlem (process) olarak çalıştığı, kod tabanının zamanla yönetilemez hale geldiği ve yatay ölçeklendirmenin (horizontal scaling) donanım israfına yol açtığı bir yapıdır.

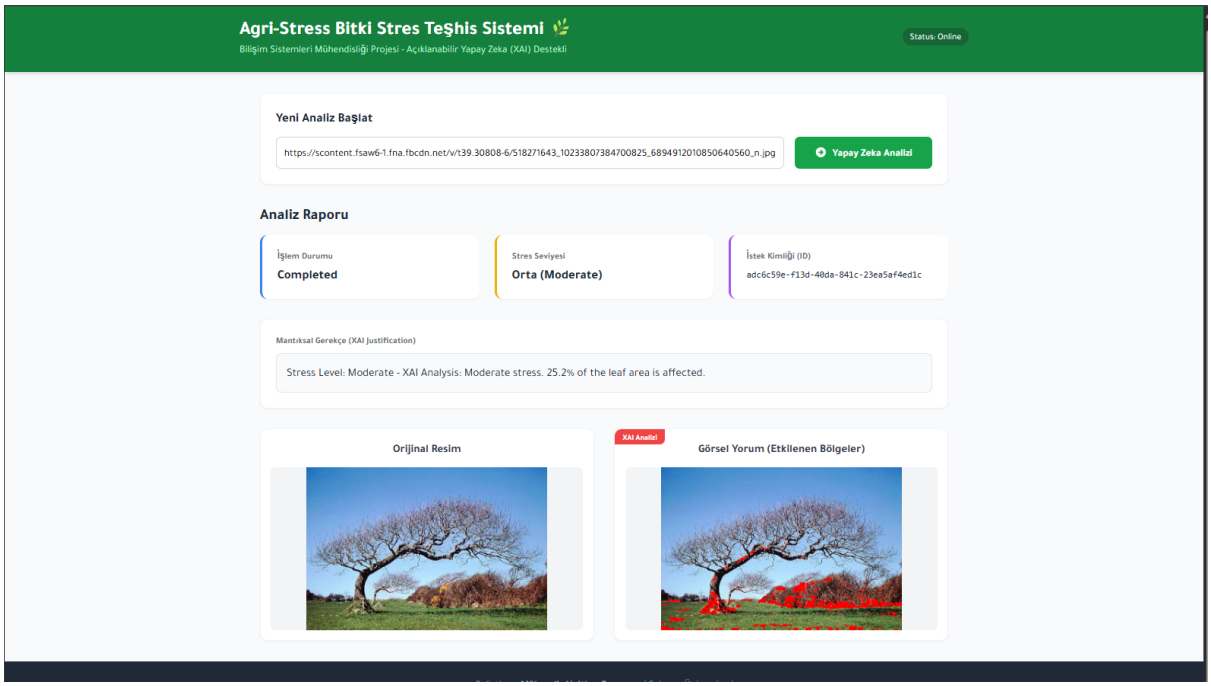
Bu projenin temelini oluşturan Mikro Hizmet (Microservices) Mimarisi ise; büyük bir sistemi, her biri kendi iş alanından (domain) sorumlu, bağımsız olarak geliştirilip dağıtılabilen (independently deployable) ve HTTP/REST protokolleri ile haberleşen küçük, özerk servislere böler. **Alshuqayran, Ali ve Evans (2023) tarafından yapılan kapsamlı araştırmada da vurgulandığı üzere, mikro hizmet mimarisi esneklik sağlarken, taşıma (migration) ve sistem tasarımı açısından çeşitli zorlukları da beraberinde getirmektedir.** Bu proje, bitki stresini teşhis eden karmaşık bir yapay zeka modelini (AI) ve Açıklanabilir Yapay Zeka (XAI) algoritmalarını, bulut bilişim standartlarında, yüksek erişilebilir bir mikro hizmet altyapısında sunmayı amaçlamaktadır.

2. Sistem Mimarisi ve Bileşenlerin Derinlemesine Analizi



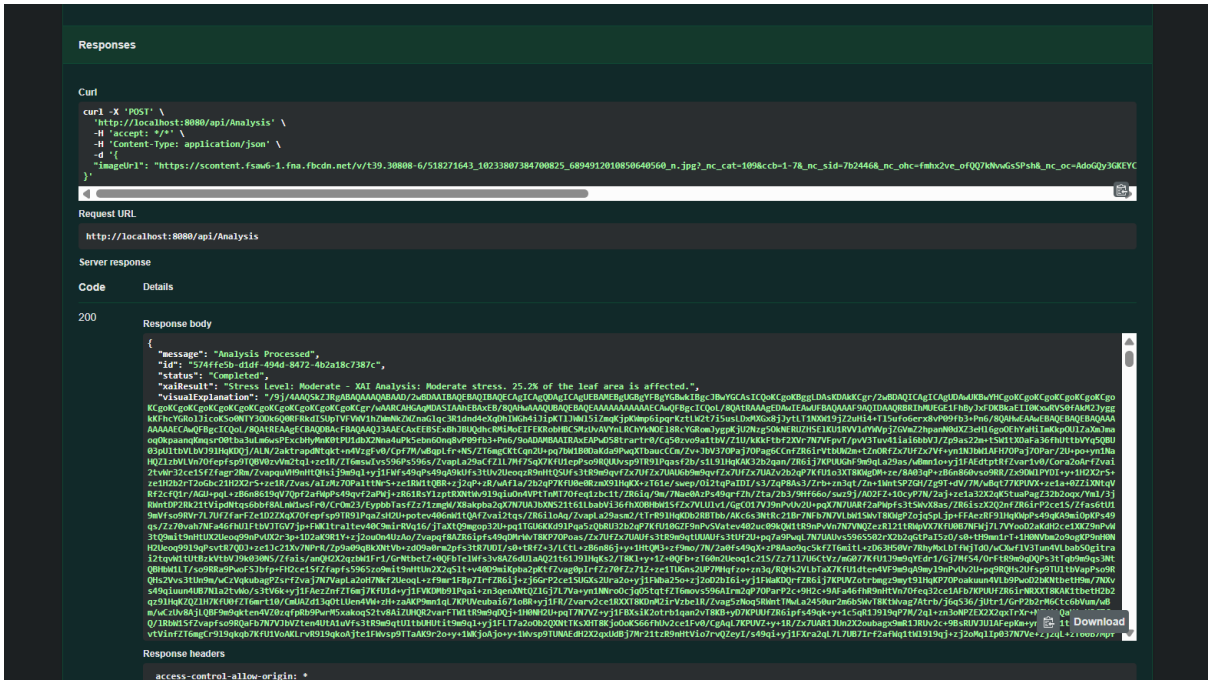
Agri-Stress sistemi, "Gevşek Bağlı" (Loosely Coupled) prensibine göre tasarlanmış 5 ana katmandan oluşmaktadır:

2.1. Sunum Katmanı (Frontend)



Kullanıcıların (çiftçiler/uzmanlar) sistemle etkileşime girdiği asenkron web arayüzüdür. Kullanıcı bir bitki fotoğrafı bağlantısı yüklediğinde, arayüz arka planda (AJAX/Fetch API ile) API Gateway'e istek atar ve sonucu beklerken arayüz kilitlenmez (non-blocking). Dönen sonuçta sadece teşhis değil, aynı zamanda yapay zekanın odaklandığı noktaları gösteren XAI Isı Haritası (Heatmap) gösterilir.

2.2. API Ağ Geçidi ve Orkestratör (.NET 10)


Responses
Curl
curl -X 'POST' \
'http://localhost:8080/api/Analysis' \
-H 'accept: */*' \
-H 'Content-Type: application/json' \
-d '{
"imageUrl": "https://scontent.fsaw6-1.fna.fbcdn.net/v/t39.30808-6/518271643_1023380738470082 5_6894912010850640560_n.jpg?_nc_cat=109&ccb=1-7&_nc_sid=7b2446&_nc_ohc=fmhx2ve_ofQQ7kNvwGs5Psh&_nc_oc=AdoGQy3GKEYC
}'
Request URL
http://localhost:8080/api/Analysis
Server response
Code
Details
200
Response body
{
"message": "Analysis Processed",
"id": "574ffe5b-d1df-494d-8472-4b2a18c7387c",
"status": "Completed",
"xaiResult": "Stress Level: Moderate - XAI Analysis: Moderate stress. 25.2% of the leaf area is affected.",
"visualExplanation": "/9j/4AAQSkZJRgABAQAAAQABAAD/2wBDAAIBAQEBAQIBAQECAgICAgQDAgICAgUEBAMEBgUGBgYFBgYGBwkIBgcJBwYGCAsICQoKCgoKBggLDAsKDAkKCgr/2wBDAQICAgICAgUDAwUKBwYHCgoKCgoKCgoK

- **Teknoloji: Yüksek performanslı ve çok iş parçacıklı (multi-threaded) yapısı nedeniyle C# .NET 10 kullanılmıştır.**
- **İş Akışı: Gelen POST isteklerini alır, doğrular (Validation), veriyi Python AI servisine yönlendirir, oradan dönen analiz sonucunu PostgreSQL'e kaydeder ve Frontend'e nihai JSON formatında 200 OK yanıtı döner.**
- **Swagger (OpenAPI Spesifikasyonu): API uç noktalarının dokümantasyonu ve test edilebilirliği için sisteme Swashbuckle kütüphanesi üzerinden Swagger entegre edilmiştir. Swagger, sadece bir kullanıcı arayüzü değil, aynı zamanda sistemin API sözleşmesini (Contract) belirleyen standart bir makine okuyabilir belgedir (OpenAPI JSON). Bu sayede frontend geliştiricileri API'nin tam olarak ne beklediğini ve ne döneceğini bilir.**

2.3. Yapay Zeka ve XAI Motoru (Python FastAPI)

- **Teknoloji: Derin öğrenme ve veri bilimi kütüphanelerinin (TensorFlow, PyTorch) ana dili olan Python ve yüksek hızlı web çatısı FastAPI kullanılmıştır.**
- **Algoritma (XAI):** Evrişimli Sinir Ağları (CNN) kullanılarak hastalık teşhisi yapılır. **Raza vd. (2024) ve Shah vd. (2025) yaptıkları çalışmalarda, bitki yaprak hastalıklarının tespitinde derin öğrenme ile entegre edilmiş XAI yaklaşımlarının, model güvenilirliğini büyük ölçüde artırdığını kanıtlamışlardır. Benzer şekilde Islam ve Hassan (2023), tarımsal yapay zeka modellerinde şeffaflığın sağlanmasının son kullanıcı güveni için hayati olduğunu belirtmektedir.** Bu doğrultuda sistemimiz sadece sonuç üretmekle kalmaz, "Yapay zeka bu kararı neden verdi?" sorusunu cevaplamak için Grad-CAM (Gradient-weighted Class Activation Mapping) algoritmasını çalıştırır. **Thaduri vd. (2024) tarafından sunulan çalışmada Grad-CAM tabanlı yaklaşımların, kenar cihazlarda (edge) hastalık sınıflandırması yaparken hedef bölgelere odaklanmayı (mask-guided attention) başarılı bir şekilde sağladığı gösterilmiştir.** Agri-Stress sistemi de bu literatürü temel alarak, yaprak üzerindeki sorunlu pikselleri kırmızı renkle vurgulayan bir Isı Haritası (Heatmap) üreterek modelin şeffaflığını (Transparency) sağlar.
- 2.4. Veri Depolama (PostgreSQL & EF Core)

Kalıcı verilerin (Persistent Data) tutulduğu ilişkisel veri tabanıdır. .NET API, bu veri tabanına doğrudan SQL yazmak yerine modern bir ORM olan Entity Framework Core (EF Core) kullanarak iletişim kurar. Bu sayede veri tabanı şeması tamamen kod (Code-first) üzerinden yönetilir.

3. Konteynerizasyon ve Docker Mimari Kararları

Sistemin ortam bağımsız (Immutable Infrastructure) çalışabilmesi için her servis Dockerize edilmiştir.

3.1. Çok Aşamalı İnşa (Multi-stage Build - .NET API)

API imajının güvenliğini artırmak ve boyutunu küçültmek için bu strateji kullanılmıştır:

- **Build Aşaması: Ağır SDK imajı (mcr.microsoft.com/dotnet/sdk:10.0) çekilir, projeler derlenir (dotnet build) ve yayınlanır (dotnet publish).**
- **Runtime Aşaması: Derleyici içermeyen çok hafif bir çalışma zamanı imajı (aspnet:10.0) alınır ve sadece derlenmiş ikili dosyalar (.dll) bu imaja kopyalanır. Sonuç olarak kaynak kodlar imaj içinde yer almaz (Güvenlik) ve imaj boyutu minimuma iner (Performans).**

3.2. Docker Compose ve Bağımlılık Yönetimi

docker-compose.yml dosyası, tüm servislerin aynı sanal köprü ağında (Bridge Network) DNS üzerinden haberleşmesini sağlar. Sistem başlatılırken API'nin Veri Tabanından önce açılıp çökmesini (Race Condition) engellemek için, PostgreSQL servisine bir "Healthcheck" (Sağlık Kontrolü) tanımlanmış ve API'nin depends_on kuralı condition: service_healthy olarak ayarlanmıştır.

4. Bulut Orkestrasyonu: Kubernetes (K8s) Bildirimleri

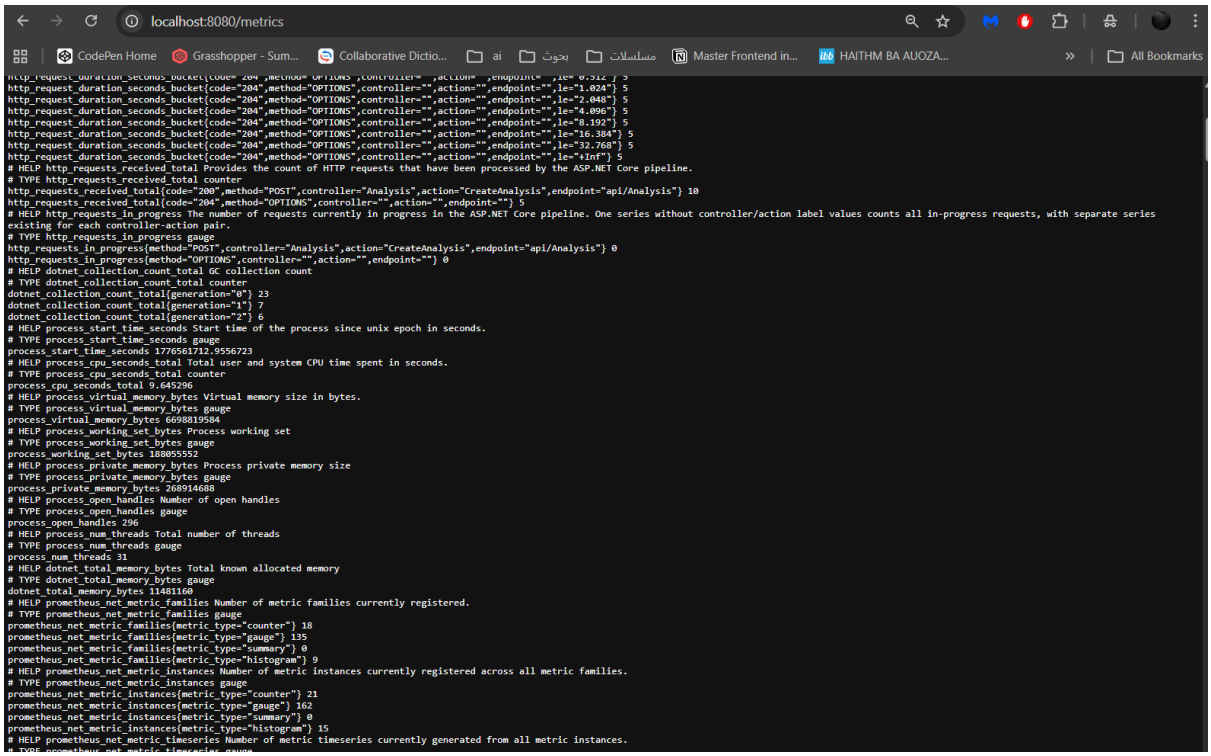
Sistemin yerel ortamdan üretim (Production) ortamına taşınması ve yüksek erişilebilirlik şartlarının sağlanması için Kubernetes manifestoları (.yaml) kullanılmıştır.

- **Deployments & Replicas: API servisi için 3 Replika, yoğun kaynak tüketen Vision servisi için 2 Replika tanımlanmıştır. Bu sayede Kubernetes'in Yük Dengeleyicisi (Load Balancer) gelen istekleri bu pod'lar arasında dağıtır.**
- **Self-Healing (Kendi Kendini İyileştirme): Pod'lar herhangi bir nedenle çökerse (CrashLoopBackOff veya OOMKilled), Kubernetes'in kontrol döngüsü (Control Loop) bunu anında fark eder ve istenen replika sayısını korumak için yeni bir Pod başlatır.**
- **Servis İzolasyonu: Güvenlik mimarisi gereği, Veri tabanı ve Vision servisi ClusterIP ile sadece küme (Cluster) içi iletişime sınırlandırılmış; sadece .NET API servisi dış dünyaya açılmıştır.**
- **Kaynak Sınırları (Resource Limits): Python AI konteynerinin kümedeki (Cluster) tüm RAM'i tüketmesini önlemek için, Pod tanımlamalarına kesin CPU ve Bellek (Memory) kısıtlamaları getirilmiştir.**

5. Gözlemlenebilirlik: Prometheus ve Grafana Entegrasyonu

Mikro hizmet mimarilerindeki en büyük zorluk olan sistem izlenebilirliği (Observability), sektör standartları olan Prometheus ve Grafana ile çözülmüştür.

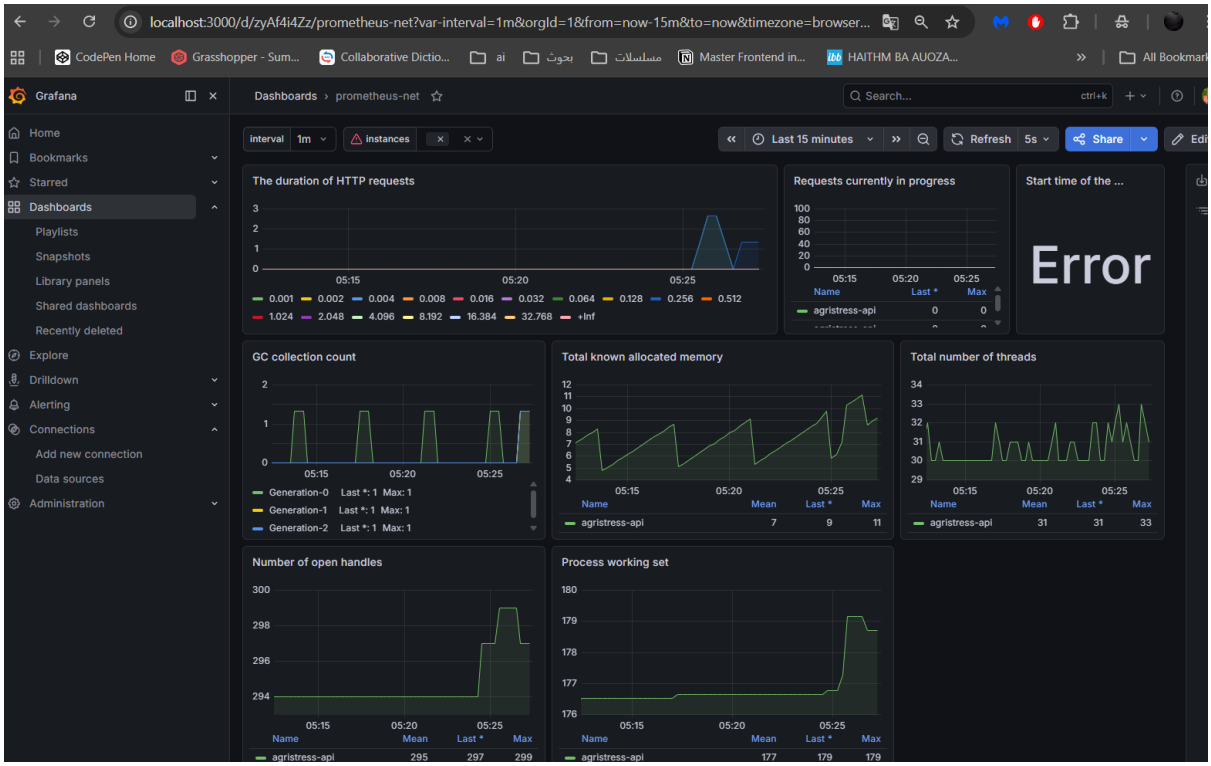
5.1. Prometheus Metrikleri ve Matris Veri Modeli (Matrix Data Model)



Prometheus, klasik loglama araçlarının aksine, verileri "Zaman Serisi Matrisi" (Time-Series Matrix Data Model) olarak saklar. Bu modelde metrikler, zaman damgaları ve anahtar-değer (Key-Value) etiketleriyle çok boyutlu olarak tutulur. Sistemimizde Prometheus (prometheus.yml), .NET API'nin sunduğu /metrics uç noktasını (Endpoint) her 20 saniyede bir dinleyerek (Pull yöntemi) veri çeker. Toplanan kritik metrikler şunlardır:

- **process_start_time_seconds: Uygulamanın başlatılma zamanını saniye cinsinden tutar.**
- **http_requests_total: Gelen HTTP isteklerinin toplam sayısını durum kodlarına (200, 404, 500) göre ayrıştırarak tutar.**
- **process_working_set_bytes: Konteynerların kullandığı aktif fiziksel bellek miktarını izler.**

5.2. Grafana Gösterge Paneli (Dashboard)



Grafana, Prometheus'un topladığı bu karmaşık matris verilerini PromQL (Prometheus Query Language) kullanarak sorgular ve sistem yöneticisinin anlayabileceği görsel panellere dönüştürür. Panellerimiz şunları içerir:

- **Uptime (Çalışma Süresi) Paneli: time() - process_start_time_seconds{job="agristress-api"} sorgusu ile API'mizin ne kadar süredir kesintisiz ayakta olduğunu gösterir.**
- **HTTP Request Rate: Saniyede gelen istek oranını izleyerek sisteme binen anlık yükü tespit eder.**
- **Memory Usage & Garbage Collection: AI modellerinin çalışması sırasında oluşan RAM tüketim piklerini ve .NET'in çöp toplayıcısının (GC) belleği ne kadar etkili temizlediğini canlı olarak gösterir.**

6. Ölçeklendirme Stratejisi (Scaling Strategy)

Mikro hizmet mimarilerinde otomatik ölçeklendirme sistemlerinin (automatic scaling systems) bulut bilişimdeki çevrimiçi uygulamalar için hayati olduğu Wang vd. (2022) tarafından yapılan çalışmalarda vurgulanmaktadır. Sistemimiz de durumsuz (Stateless) yapısı sayesinde Kubernetes üzerinde Yatay Ölçeklendirme (Horizontal Scaling / Scale Out) kapasitesine tam sahiptir.

Kullanıcı sayısı aniden arttığında (Örn: Hasat mevsiminde sisteme binen yük), Kubernetes HPA (Horizontal Pod Autoscaler) modülü devreye girer. **Kubernetes ortamında mikro hizmetlerin trafiğe göre dinamik olarak ölçeklendirilmesinin (autoscaling) performansı ve kaynak kullanımını optimize ettiği Joyce ve Sebastian (2022) tarafından da desteklenmektedir.** Sistemimizde CPU kullanımı belirlenen eşiği (Örn: %80) aştığında, .NET API'nin replika sayısını otomatik olarak 3'ten 15'e çıkarabilir. Yük bittiğinde ise tekrar 3'e düşürerek gereksiz bulut sunucu (Cloud Billing) maliyetlerini önler.

7. Sonuç ve Öğrenilen Dersler

Agri-Stress projesi, teorik mikro hizmet ve konteynerizasyon konseptlerinin, XAI (Açıklanabilir Yapay Zeka) gibi ağır endüstriyel modellere nasıl başarılı bir şekilde entegre edilebileceğini kanıtlamıştır. Proje geliştirme sürecinde öğrenilen en önemli ders; sistemi mikro parçalara bölmenin kod yazmayı kolaylaştırdığı ancak Ağ Yönetimi (Networking) ve Gözlemlenebilirlik (Monitoring) konularında yeni operasyonel karmaşıklıklar getirdiğidir. Docker ve Kubernetes kullanımı, sistemin "Benim bilgisayarımda çalışıyordu" probleminden kurtularak tam anlamıyla "Üretime Hazır" (Production-Ready) bir bulut mimarisine dönüşmesini sağlamıştır.

8. Kaynakça

Raza, A., Hu, J., Ali, Y., & Baig, M. J. A. (2024). Plant leaf disease detection using ensemble learning and explainable AI. *IEEE Access*, *12*. https://ieeexplore.ieee.org/abstract/document/10729231

Wang, J., et al. (2022). An automatic scaling system for online application with microservices architecture. In *2022 IEEE International Conference on Joint Cloud Computing (JCC)*. IEEE. https://ieeexplore.ieee.org/document/9898085

Alshuqayran, N., Ali, N., & Evans, R. (2023). A survey on microservices architecture: Principles, patterns and migration challenges. *IEEE Access*, *11*. https://ieeexplore.ieee.org/document/10220070

Islam, M. S., & Hassan, M. M. (2023). Explainable AI for deep learning based potato leaf disease detection. In *2023 IEEE International Conference on Fuzzy Systems (FUZZ)*. IEEE. https://ieeexplore.ieee.org/document/10309803

Shah, P., Punia, D., Sharma, A. K., & Sikka, G. (2025). Plant disease detection using ensemble technique and explainable AI. In *2025 IEEE International Conference on Electronics, Computing and Communication Technologies*. IEEE. https://ieeexplore.ieee.org/document/11101497

Thaduri, A., Jog, M. C., Velmurugan, R., & Baghini, M. S. (2024). Novel Grad-CAM based mask-guided attention for plant disease classification on edge. In *2024 IEEE International Conference on Omni-layer Intelligent Systems (COINS)*. IEEE. https://www.researchgate.net/publication/383168070_Novel_Grad-CAM_based_Mask-Guided_Attention_for_Plant_Disease_Classification_on_Edge

Joyce, J. E., & Sebastian, S. (2022). Reinforcement learning based autoscaling for Kafka-centric microservices in Kubernetes. In *2022 IEEE 4th PhD Colloquium on Emerging Domain Innovation and Technology for Society (PhD EDITS)*. IEEE. https://ieeexplore.ieee.org/document/9955300

Note* Yapay zeka tarafından üretildi